PARQUEADERO CON PATRÓN MEMENTO

Patrón de Diseño Comportamental: *Memento*

Control de estados, cálculo de tarifas y restauración de tickets para vehículos

Autor: **Angie Nathaly Cisneros Guerrero**

Código: **20242020084**

Fecha: Octubre 2026

Índice

1. Introducción **2**

2. Objetivos **3**
2.1. Objetivo General 3
2.2. Objetivos Específicos 3

3. Diagrama UML **4**

4. Explicación de la Funcionalidad de las Clases **5**
4.1. Clase Abstracta Originator (Vehiculo.java) 5
4.2. Clases Concretas (Carro.java, Moto.java, Bus.java) 8
4.3. Clase Memento (Memento.java) 8
4.4. Clase Administradora (Caretaker.java) 10

5. Código Cliente **10**

6. Pruebas y Ejecución **12**
6.1. Prueba de Ejecución en Consola 12

7. Repositorio **13**
7.1. Enlace GitHub 13

1. Introducción

En los sistemas de parqueadero, el registro de los tiempos de ingreso, salida y el cálculo de tarifas de acuerdo con el tipo vehicular (carros, motos, buses) exigen un almacenamiento de historial de los estados del vehículo para auditorías o reversiones de operaciones sin romper el principio de encapsulamiento.

Para dar solución se hace la implementación del patrón de diseño comportamental **Memento**.permitiendo capturar y externalizar el estado interno de un objeto (Originator) vehiculo, sin violar la encapsulación, de forma que el vehículo pueda volver a dicho estado (restaurar un ticket o cobro previo) más adelante mediante un administrador de historial (*Caretaker*).

2. Objetivos

2.1. Objetivo General

Implementar un sistema polimórfico de gestión de parqueadero aplicando el patrón de diseño **Memento** para restaurar los estados temporales de vehículos.

2.2. Objetivos Específicos

- Diseñar una clase abstracta base `Vehiculo` que centralice los atributos comunes y declare métodos abstractos de cobro para garantizar el polimorfismo.
- Implementar subclases específicas (`Carro`, `Moto`, `Bus`) con tarifas e identidades propias.
- Desarrollar la clase inmutable `Memento` para encapsular los estados de tiempos y pagos de manera aislada.
- Construir la clase `Caretaker` para gestionar el almacenamiento en memoria de una lista ordenada de respaldos o tickets.

3. Diagrama UML

Descripción General

El diagrama de clases muestra la estructura del sistema. La clase abstracta `Vehiculo` actúa como el *Originator* y es extendida por `Carro`, `Moto` y `Bus`. La clase `Memento` almacena el estado interno de los vehículos de forma inmutable, mientras que la clase `Caretaker` administra una colección de dichos mementos.

Diagrama de Clases

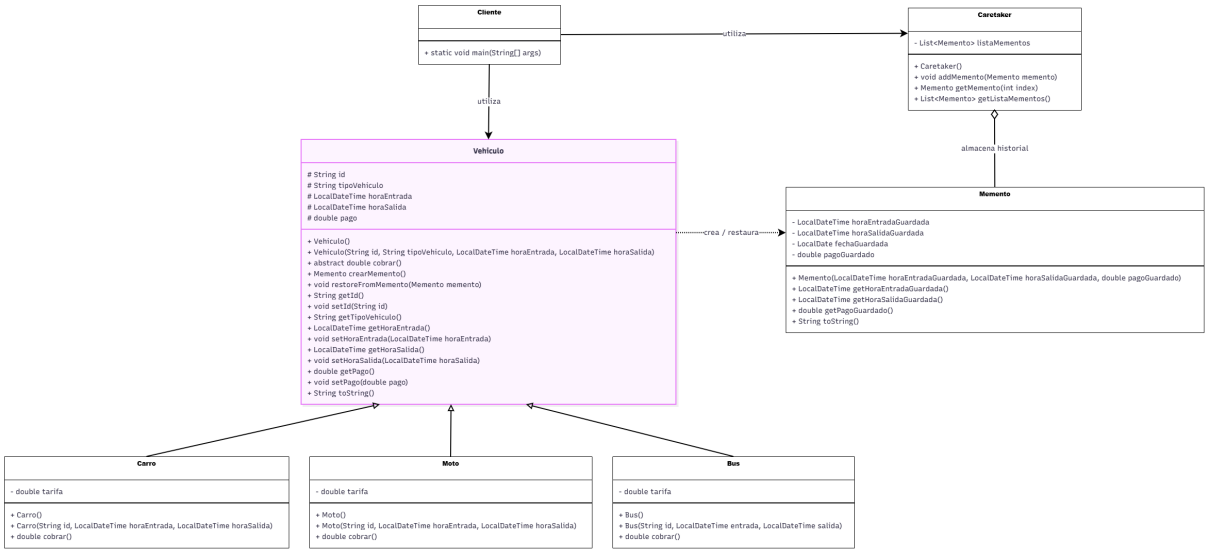


Figura 1: Diagrama de Clases del Sistema de Parqueadero con Patrón Memento.

4. Explicación de la Funcionalidad de las Clases

A continuación se detalla el rol y la estructura de cada uno de los componentes

4.1. Clase Abstracta Originator (Vehiculo.java)

Define los atributos comunes a cualquier vehiculo (placa o ID, tipo de vehículo, fecha/hora de entrada y salida, y pago acumulado). Cuenta con el método abstracto `cobrar()` que obliga a las subclases a calcular la tarifa correspondiente, además de los métodos para crear y restaurar mementos.

```
1  package parqueadero.memento;
2  import java.time.LocalDateTime;
3
4  /**
5   *
6   * @author natha
7   */
8  public abstract class Vehiculo {
9
10     //Estados
11
12     protected String id;
13     protected String tipoVehiculo;
14     protected LocalDateTime horaEntrada;
15     protected LocalDateTime horaSalida;
16     protected double pago;
17
18     //Constructores
19
20     public Vehiculo() {
21             this.pago = 0;
22     }
23
24     public Vehiculo(String id, String tipoVehiculo, LocalDateTime
          horaEntrada, LocalDateTime horaSalida) {
25          this.id = id;
26          this.tipoVehiculo = tipoVehiculo;
27          this.horaEntrada = horaEntrada;
28          this.horaSalida = horaSalida;
29          this.pago = 0;
30     }
31
32     //Metodo calcular pago
33     public abstract double cobrar();
34
35     //metodo guardar estados
36     public Memento crearMemento(){
37
38         Memento mementoGuardado = new Memento (this.horaEntrada,
              this.horaSalida, this.pago);
```

```
39          return mementoGuardado;
40      }
41
42      //Metodo restaurar estados
43      public void restoreFromMemento(Memento memento){
44
45          if (memento != null) {
46              this.horaEntrada = memento.getHoraEntradaGuardada();
47              this.horaSalida = memento.getHoraSalidaGuardada();
48              this.pago = memento.getPagoGuardado();
49          }
50
51      }
52
53      //Getters
54
55      public String getId() {
56          return id;
57      }
58
59      public String getTipoVehiculo() {
60          return tipoVehiculo;
61      }
62
63      public LocalDateTime getHoraEntrada() {
64          return horaEntrada;
65      }
66
67      public LocalDateTime getHoraSalida() {
68          return horaSalida;
69      }
70
71      public double getPago() {
72          return pago;
73      }
74
75      //Seters
76
77      public void setId(String id) {
78          this.id = id;
79      }
80
81      public void setTipoVehiculo(String tipoVehiculo) {
82          this.tipoVehiculo = tipoVehiculo;
83      }
84
85      public void setHoraEntrada(LocalDateTime horaEntrada) {
86          this.horaEntrada = horaEntrada;
87      }
88
89      public void setHoraSalida(LocalDateTime horaSalida) {
```

```
90          this.horaSalida = horaSalida;
91      }
92
93      public void setPago(double pago) {
94          this.pago = pago;
95      }
96
97      //toString
98
99  @Override
100     public String toString() {
101         return "\n========================================\n" +
102                "␣␣␣␣␣␣␣␣␣␣TICKET␣DE␣PARQUEADERO␣␣␣␣␣␣␣␣␣␣\n" +
103                "========================================\n" +
104                "␣Tipo␣de␣Vehiculo␣:␣" + tipoVehiculo + "\n" +
105                "␣Placa␣:␣" + id + "\n" +
106                "␣Hora␣de␣Entrada␣␣:␣" + (horaEntrada != null ?
                     horaEntrada : "No␣registrada") + "\n" +
107                "␣Hora␣de␣Salida␣␣␣:␣" + (horaSalida != null ?
                     horaSalida : "En␣curso...") + "\n" +
108                "␣----------------------------------------\n" +
109                "␣TOTAL␣A␣PAGAR␣␣␣␣:␣$" + String.format("%.2f", pago
                     ) + "\n" +
110                "========================================\n";
111     }
112 }
```

4.2. Clases Concretas (Carro.java, Moto.java, Bus.java)

Hrederán de `Vehiculo` implementando tarifas específicas por minuto para cada categoría vehicular.

```java
package parqueadero.memento;

import java.time.Duration;
import java.time.LocalDateTime;

/**
 *
 * @author natha
 */
public class Carro extends Vehiculo{

    private static final double tarifa = 5000;

    //Constructor

    public Carro() {
    }

    public Carro(String id,  LocalDateTime horaEntrada,
       LocalDateTime horaSalida) {
        super(id,"CARRO", horaEntrada, horaSalida);
    }

    @Override
    public double cobrar() {
        if (horaEntrada != null && horaSalida != null) {
            long minutos = Duration.between(horaEntrada, horaSalida
               ).toMinutes();
            this.pago = (minutos) * tarifa;
        }
        return this.pago;
    }

}
```

4.3. Clase Memento (Memento.java)

Encapsula el estado interno del vehículo en un momento dado de manera inmutable (con atributos `final`), previniendo alteraciones externas.

```java
package parqueadero.memento;

import java.time.LocalDate;
import java.time.LocalDateTime;

/**
 *
```

```
 * @author natha
 */
public class Memento {

    // Estados Guardados

    private final LocalDateTime horaEntradaGuardada;
    private final LocalDateTime horaSalidaGuardada;
    private LocalDate fechaGuardada;
    private final double pagoGuardado;

    //Constructor

    public Memento(LocalDateTime horaEntradaGuardada, LocalDateTime
        horaSalidaGuardada,double pagoGuardado) {
        this.horaEntradaGuardada = horaEntradaGuardada;
        this.horaSalidaGuardada = horaSalidaGuardada;
        this.pagoGuardado = pagoGuardado;
    }

    //Getters

    public LocalDateTime getHoraEntradaGuardada() {
        return horaEntradaGuardada;
    }

    public LocalDateTime getHoraSalidaGuardada() {
        return horaSalidaGuardada;
    }

    public double getPagoGuardado() {
        return pagoGuardado;
    }

    //toString

    @Override
    public String toString() {
        return
                "\n=========================================\n" +
                "␣␣␣␣␣␣␣␣␣␣␣␣TICKET␣MEMENTO␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n" +
                "=========================================\n" +
                "␣Entrada␣Guardada␣:␣" + horaEntradaGuardada + "\n"
                   +
                "␣Salida␣Guardada␣␣␣:␣" + horaSalidaGuardada + "\n" +
                "␣Pago␣Acumulado␣␣␣:␣$" + String.format("%.2f",
                   pagoGuardado) + "\n" +
                "=========================================";
    }

}
```

4.4. Clase Administradora (Caretaker.java)

Se encarga de almacenar y custodiar los objetos `Memento` en una lista dinámica, permitiendo recuperar estados anteriores según un índice.

```java
package parqueadero.memento;

import java.util.ArrayList;
import java.util.List;

/**
 *
 * @author natha
 */
public class Caretaker {

    private List<Memento> listaMementos;

    //Constructor

    public Caretaker() {
        this.listaMementos = new ArrayList<Memento>();
    }

    //Anade Mementos
    public void addMemento(Memento memento){
        listaMementos.add(memento);
    }

    //Retorna un Memento especifico
    public Memento getMemento(int index){
        return listaMementos.get(index);
    }

    //Retorna el historial de mementos
    public List<Memento> getListaMementos() {
        return listaMementos;
    }

}
```

5. Código Cliente

La clase `Cliente` simula el flujo completo de creación de un vehículo, el respaldo de su estado inicial, la modificación del tiempo de salida, el cálculo de cobro y la posterior restauración del estado previo.

```java
package parqueadero.memento;

import java.time.LocalDateTime;

```

```java
/**
 *
 * @author natha
 */
public class Cliente {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {

        Caretaker caretaker = new Caretaker();

        Vehiculo objCarro = new Carro("ABC-123", LocalDateTime.of
           (2026, 10, 23, 14, 30), LocalDateTime.of(2026, 10, 23,
           15, 0));
        System.out.println(objCarro);

        System.out.println("Guardando memento 1");
        Memento memento1 =objCarro.crearMemento();
        caretaker.addMemento(memento1);

        System.out.println("Cambianfo hora salida y cobrando");
        objCarro.setHoraSalida(LocalDateTime.of(2026, 10, 23, 16,
           0));
        objCarro.cobrar();
        System.out.println(objCarro);

        System.out.println("Guardando memento 2");
        Memento memento2 =objCarro.crearMemento();
        caretaker.addMemento(memento2);
        System.out.println(memento2.toString());

        System.out.println("Restaurando memento 1:");
        objCarro.restoreFromMemento(memento1);
        System.out.println(objCarro.toString());



    }

}
```

6. Pruebas y Ejecución

6.1. Prueba de Ejecución en Consola

Salida de consola cuando se ejecuta el cliente
Prueba Cliente

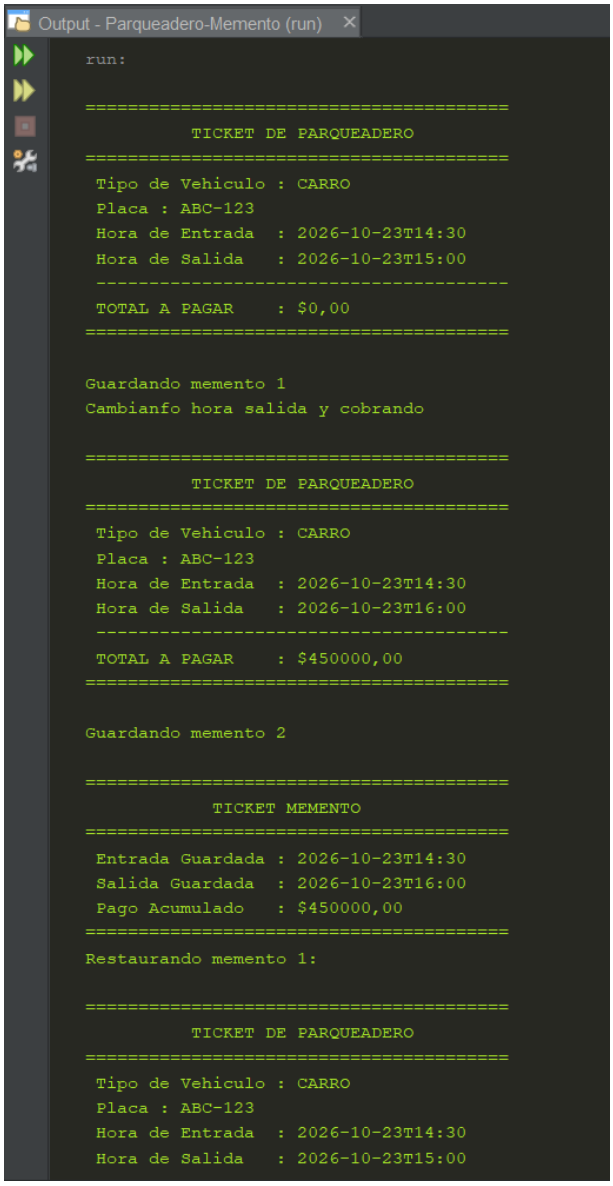


Figura 2: Salida en consola.

7. Repositorio

7.1. Enlace GitHub

- https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20Comportamiento/Parqueadero-Memento